David Gutierrez
Dr Mohammad Tariqul Islam
CSC 461
May 12, 2025

Article Writing (Low-Rank Approximation (LoRA))

Modern generative AI spans and encompasses many fields/subjects including computer vision, natural language processing and in particular this paper, *Low Rank Approximation for Learned Query Optimization,* by Zixuan Yi, Yao Tian, Zachary G. Ives, and Ryan Marcus, explores how recent advancements to machine learning to completely replace or aid a traditional query optimizer have their drawbacks and how to possibly address these problems using LoRa. It explains how the nature of "learning techniques can cause unpredictable regressions, (e.g. "My query was fast yesterday, why is it slow today?")", suffer from expensive overhead costs like training and inference. As well as how they often make assumptions about the underlying database management system(DBMS),"such as the availability of features or the structure of query plans". This paper addresses these current limitations by proposing a low overhead cost, reliable alternative based on low-rank approximation (LoRA), called LimeQO.

Large language models like Chat GPT or LLaMA have been essential in modern natural language processing applications. However, fully fine tuning these large models are computationally extremely expensive and are often infeasible for many users due to various limitations. Low Rank Approximation (LoRA), offers an efficient fine tuning method that can drastically reduce the number of trainable parameters being addressed while maintaining a competitive performance for the model. This leverages the idea that weight updates during fine tuning often lie in a low dimensional subspace and instead of modifying the original large weight matrices of an LLM, LoRA introduces low rank decomposition matrices that approximate the changes only needed for the model.

The author's proposal of LimeQO, uses LoRa fine tuning to minimize offline resources usage while trying to maximize performance improvements. While the authors' framework (LimeQO) can take advantage of current modern expensive neural networks, LimeQO is trained with purely linear methods(mostly matrix completion), which can lead to drastically "simpler and lower overhead implementation". " LimeQO leverages matrix completion to estimate the performance of unobserved query plans. By modeling the optimization problem as a matrix of query/hint combinations". As a result, LimeQO avoids expensive training, makes no assumptions about internal DBMS structures,"except that each query plan has a number of alternatives with measurable latency".

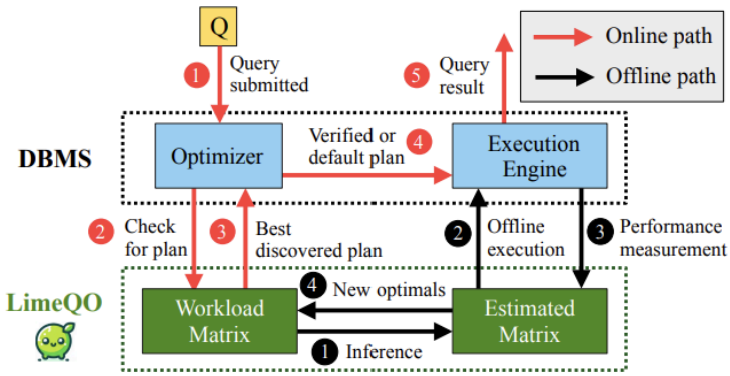


Figure 2: LimeQO system model

LimeQO model consists of two main paths: an online path, in which user submitted queries are executed using plans that have been verified to be fast, and an offline path, where LimeQO can perform offline exploration. "In the online path, 1(image (red like picture)) user-submitted queries are received by the DBMS' traditional optimizer. Then, 2(image) the optimizer asks LimeQO if a better query plan has been observed for this query. 3(image) LimeQO replies with either a query plan that is faster than the default plan, or the default plan. 4(image) This verified plan is then executed, 5(image) and the results are returned. In the offline path, LimeQO searches for better query plans." The offline search can happen when the database management system are idle or while being performed on a snapshot of the database. "During this time, LimeQO will 1(image) predict the performance of all query plans in the workload matrix, and then 2(image) select the most promising query plans. 3(image) These promising plans are executed, 4(image) and their performance is recorded. 5(image) The now-observed values are then stored in the workload matrix." This workflow is based on the assumption that the query workload matrix is low rank and if it was a high rank one, the authors stated that, "it is unlikely that matrix completion will make accurate predictions for unobserved plans", meaning it would most likely fail.

However by using LoRa, the authors were able to achieve a significant improvement in speed. There was a two times speed up in total query latency after just a few hours of offline exploration. It even performed comparable to deep learning models but used up to 100x less computational resources. As well as "guarantees no performance regressions because it only deploys plans that have been empirically verified to outperform the default. In contrast, neural network–based approaches, such as those using Tree Convolutional Neural Networks (TCNNs), may select suboptimal plans based on model predictions, which can lead to unpredictable slowdowns in query performance". Main problem with this newer model of using LoRa despite all of the strengths, LimeQO are limited to repetitive workloads, and cannot infer for unseen queries without having them added to the matrix beforehand.